

Deep Research: Avatrada 57 Topic 052

Engineering Report: Automated SSH Key Rotation Policy

Report ID: EDR-2023-052-SSH **Author:** Autonomous Technical Researcher
Subject: Deep-Dive Analysis of an Automated SSH Key Rotation System

Executive Summary

This report provides a detailed engineering analysis of the specified "Automated 90-day rotation of SSH keys" policy. The analysis deconstructs the core components of such a system, presents concrete implementation strategies using both a standalone shell script and a more robust Ansible playbook, and performs a critical analysis of potential failure modes, security vulnerabilities, and scalability optimizations.

The primary objective is to create a reliable, automated process that enhances security by limiting the lifetime of SSH credentials. The recommended approach for scalable and manageable infrastructure is the Ansible-based solution due to its idempotency, state management, and integration with secure secret storage like Ansible Vault. For simpler environments, a well-designed cron job executing a robust script is a viable alternative. The ultimate evolution of this policy for enterprise environments involves transitioning to an SSH Certificate Authority (CA) model, which is also discussed as a long-term optimization.

1. Technical Deconstruction

The system can be broken down into five core components: server-side hardening, key generation, secure key distribution, connection verification, and old key decommissioning.

1.1. Server-Side Hardening (`sshd_config`)

The foundation of this policy is ensuring the target server only accepts public key authentication. This is a critical security measure that prevents brute-force password attacks.

- **Mechanism:** The OpenSSH server daemon (`sshd`) is configured via the `/etc/ssh/sshd_config` file.
- **Key Directives:**
 - `PasswordAuthentication no` : This directive explicitly disables all forms of interactive password-based authentication.
 - `PubkeyAuthentication yes` : This ensures that public key authentication is enabled. This is the default on most systems but should be explicitly verified.
 - `ChallengeResponseAuthentication no` : Disables challenge-response authentication methods, which can sometimes be a vector for password-based logins.
 - `PermitRootLogin prohibit-password` or `no` : Prevents direct root login, forcing users to log in as an unprivileged user and escalate privileges via `sudo` .

1.2. Key Generation (`ssh-keygen`)

The process begins with the creation of a new cryptographic key pair.

- **Algorithm:** `ed25519` . This is a modern Elliptic Curve Cryptography (ECC) algorithm.
 - **Advantages:** It offers better performance and stronger security with smaller key sizes compared to older algorithms like RSA. It is resistant to many of the side-channel attacks that can affect other algorithms.
- **Mechanism:** The `ssh-keygen` command-line utility is used.
- **Command Logic:** `ssh-keygen -t ed25519 -a 100 -f /path/to/key -N ""`
 - `-t ed25519` : Specifies the key type.

- `-a 100` : Specifies 100 rounds of key derivation for passphrase protection (less relevant for non-interactive keys, but good practice).
- `-f /path/to/key` : Specifies the output file for the private key. The public key will be created at the same path with a `.pub` extension.
- `-N ""` : Provides an empty passphrase, which is necessary for automated, non-interactive processes. **This necessitates stringent protection of the private key file itself.**

1.3. Key Distribution & Authorization

The new public key must be securely placed on the server in the correct user's `authorized_keys` file.

- **Mechanism:** The file `~/.ssh/authorized_keys` on the server contains a list of public keys, one per line, that are permitted to authenticate as that user.
- **File & Directory Permissions:** For `sshd` to trust the keys, file system permissions must be strict:
 - `~/.ssh` directory: `700` (`drwx-----`)
 - `~/.ssh/authorized_keys` file: `600` (`-rw-----`)
- **Process:** The script/automaton logs into the server using the *old* key, appends the *new* public key to the `authorized_keys` file, and ensures permissions are correct.

1.4. Connection Verification

This is the most critical step to prevent self-inflicted lockouts. Before the old key is removed, the new key's functionality must be confirmed.

- **Mechanism:** A non-interactive SSH command is executed from the client to the server, explicitly instructing the SSH client to use *only* the new private key.
- **Command Logic:** `ssh -o "IdentitiesOnly=yes" -i /path/to/new_private_key user@server 'echo "Verification successful"'`
 - `-o "IdentitiesOnly=yes"` : Prevents the SSH agent from trying other keys, ensuring the test is specific to the new key.

- `-i /path/to/new_private_key`: Specifies the new private key to use for authentication.
- A successful command execution (exit code 0) confirms the new key is correctly installed and functional.

1.5. Old Key Decommissioning

Once the new key is verified, the old public key must be removed from the server's `authorized_keys` file to complete the rotation.

- **Mechanism:** A command is executed on the server (via the new or old key) to remove the specific line containing the old public key.
 - **Command Logic:** Using tools like `sed` or `grep` is common.
 - `sed -i.bak '/<content_of_old_public_key>/d' ~/.ssh/authorized_keys`
 - The `-i.bak` flag in `sed` is a safety measure that creates a backup of the file before editing it.
-

2. Implementation Strategy

Below are two distinct strategies for implementing the automated rotation policy.

2.1. Prerequisite: Server Configuration

On all target servers, ensure `/etc/ssh/sshd_config` is hardened.

```
# /etc/ssh/sshd_config

PubkeyAuthentication yes
PasswordAuthentication no
ChallengeResponseAuthentication no
PermitRootLogin no
```

After editing, reload the SSH service: `sudo systemctl reload sshd`.

2.2. Method 1: Standalone Bash Script (for Cron Job)

This script is designed to be run from a secure management host that has the initial SSH key.

```
#!/bin/bash
set -euo pipefail # Fail on error, unbound variable, or pipe failure

# --- Configuration ---
REMOTE_USER="admin"
REMOTE_HOST="server.example.com"
KEY_DIR="$HOME/.ssh/rotated_keys"
KEY_NAME="${REMOTE_USER}@${REMOTE_HOST}"
OLD_KEY_PATH="$HOME/.ssh/id_ed25519_current" # Path to the key used for this
connection
NEW_KEY_PATH="${KEY_DIR}/${KEY_NAME}_$(date +%Y%m%d%H%M%S)"

# --- Pre-flight Checks ---
if [ ! -f "${OLD_KEY_PATH}" ]; then
    echo "ERROR: Old key ${OLD_KEY_PATH} not found. Cannot connect to rotate."
    exit 1
fi
mkdir -p "$KEY_DIR"
chmod 700 "$KEY_DIR"

# --- 1. Generate New Key Pair ---
echo "Generating new ed25519 key pair: ${NEW_KEY_PATH}"
ssh-keygen -t ed25519 -a 100 -f "${NEW_KEY_PATH}" -C "${KEY_NAME}" -N ""
chmod 600 "${NEW_KEY_PATH}"
NEW_KEY_PUB=$(cat "${NEW_KEY_PATH}.pub")
OLD_KEY_PUB=$(ssh-keygen -y -f "${OLD_KEY_PATH}") # Get public key from private
key

# --- 2. Add New Public Key to Server ---
echo "Adding new public key to ${REMOTE_HOST}..."
ssh -i "${OLD_KEY_PATH}" "${REMOTE_USER}@${REMOTE_HOST}" "
    mkdir -p ~/.ssh && chmod 700 ~/.ssh;
    echo '${NEW_KEY_PUB}' >> ~/.ssh/authorized_keys;
    chmod 600 ~/.ssh/authorized_keys;
```

```

"
if [ $? -ne 0 ]; then
    echo "ERROR: Failed to add new public key to server."
    exit 1
fi

# --- 3. Verify New Key Connection ---
echo "Verifying connection with the new key..."
ssh -o "IdentitiesOnly=yes" -i "${NEW_KEY_PATH}" "${REMOTE_USER}@${REMOTE_HOST}" 'echo "SSH connection with new key successful."'
if [ $? -ne 0 ]; then
    echo "FATAL: Verification of new key failed. The old key has NOT been removed. Manual intervention required."
    exit 1
fi
echo "Verification successful."

# --- 4. Remove Old Key from Server ---
echo "Removing old public key from ${REMOTE_HOST}..."
ssh -i "${NEW_KEY_PATH}" "${REMOTE_USER}@${REMOTE_HOST}" "
    sed -i.bak \"/${$(echo ${OLD_KEY_PUB} | sed 's/\\/\\\\/\\\\\\\\\\\\/g')}/d\" ~/.ssh/
authorized_keys
"
if [ $? -ne 0 ]; then
    echo "ERROR: Failed to remove the old key. It may still be present on the server."
    # This is not a fatal error for access, but requires cleanup.
fi

# --- 5. Finalize Local Key State ---
echo "Rotation complete. Updating local current key."
mv "${NEW_KEY_PATH}" "${OLD_KEY_PATH}"
mv "${NEW_KEY_PATH}.pub" "${OLD_KEY_PATH}.pub"
echo "Old key has been replaced locally with the new key."

```

Automation with Cron:

To run this every 90 days, add an entry to the crontab (`crontab -e`):

```
# Run the SSH key rotation script at 3:00 AM on the first day of Jan, Apr, Jul,
Oct
0 3 1 1,4,7,10 * /path/to/your/rotate_ssh_key.sh >> /var/log/key_rotation.log
2>&1
```

2.3. Method 2: Ansible Playbook (Recommended for Scalability)

This approach is more robust, idempotent, and manageable across multiple hosts. It uses Ansible's built-in modules to handle state.

Playbook `rotate_ssh_key.yml` :

```
---
- name: Rotate SSH Keys for a User
  hosts: all
  become: no
  gather_facts: no

  vars:
    ansible_user: "admin" # The user whose key is being rotated
    key_name: "automated_key_for_{{ ansible_user }}"
    local_key_path: "{{ playbook_dir }}/keys/{{ inventory_hostname }}/{{
ansible_user }}_id_ed25519"

  tasks:
    - name: Ensure local directory for keys exists
      delegate_to: localhost
      file:
        path: "{{ local_key_path | dirname }}"
        state: directory
        mode: '0700'

    - name: Generate new ed25519 key pair locally
      delegate_to: localhost
      community.crypto.openssh_keypair:
        path: "{{ local_key_path }}"
        type: ed25519
```

```

    state: present
    force: yes # Overwrite existing key file to generate a new one
    mode: '0600'
    register: new_key

- name: Ensure .ssh directory exists on remote host with correct permissions
  file:
    path: "~/ssh"
    state: directory
    mode: '0700'

- name: Add the new public key to authorized_keys
  ansible.posix.authorized_key:
    user: "{{ ansible_user }}"
    key: "{{ new_key.public_key }}"
    key_options: 'from="management.host.com"' # Optional: restrict key
source
    comment: "{{ key_name }}"
    state: present
    manage_dir: no # We managed the directory in the previous step

- name: Remove all other keys managed by this playbook
  ansible.posix.authorized_key:
    user: "{{ ansible_user }}"
    key: "{{ new_key.public_key }}"
    comment: "{{ key_name }}"
    state: present
    exclusive: yes # This is the key! It removes other keys with the same
comment.

```

Execution:

1. **Install Ansible Collection:** `ansible-galaxy collection install community.crypto`.
2. **Create Inventory:** An `inventory.ini` file listing your servers.
3. **Run Playbook:** `ansible-playbook -i inventory.ini rotate_ssh_key.yml`

This playbook is superior because: * **Idempotency:** It can be run repeatedly without adverse effects. * **State Management:** The `authorized_key` module

with `exclusive: yes` intelligently manages the key, removing old ones with the same comment, which is safer than `sed`. * **No Verification Step Needed:** The Ansible model is declarative. If the playbook completes, the state is achieved. A failure in the `authorized_key` task would halt the playbook before any old key is removed, preventing lockouts.

3. Critical Analysis

3.1. Potential Failure Modes & Edge Cases

- **Script Interruption / Network Failure:**
 - **Scenario:** The script adds the new key but fails before removing the old one.
 - **Impact:** Low. The server remains accessible via both keys. The `authorized_keys` file will contain an extra key until the next successful run.
 - **Mitigation:** The Ansible playbook's atomic nature for the `authorized_key` module is more resilient.
- **Verification Failure (Script Method):**
 - **Scenario:** The new key is added, but verification fails (e.g., due to incorrect file permissions set manually, or a transient network issue).
 - **Impact:** High. The script will halt, leaving the old key in place. This is the correct "fail-safe" behavior, but it requires manual intervention to diagnose why the new key isn't working.
- **"First Key" Bootstrapping Problem:**
 - **Scenario:** How is the very first key placed on the server to enable this automated process?
 - **Impact:** This is a process dependency.
 - **Mitigation:** Initial server provisioning (e.g., via cloud-init, Kickstart, or manual setup) must place the initial "management key" on the server.

- **Loss of Management Host:**

- **Scenario:** The host running the cron job or Ansible playbook is destroyed or its private key is lost.
- **Impact:** Critical. The ability to rotate keys is lost. Access to the servers depends on the last-rotated key, which now has an indefinite lifetime.
- **Mitigation:** Implement high availability for the management host or have a documented disaster recovery procedure for its keys (e.g., backups in a secure vault).

3.2. Security Considerations

- **Private Key Security:** The entire system's security hinges on the protection of the private key on the management host.
 - **Vulnerability:** If the management host is compromised, an attacker gains access to all target servers.
 - **Mitigation (Ansible):** Use **Ansible Vault** to encrypt the generated private keys at rest.
 - **Mitigation (Cron):** Use strict filesystem permissions (`600`) and ensure the host itself is hardened and monitored. Do not store the private key in version control.
- **Audit Trail:** A simple cron job lacks a clear audit trail.
 - **Vulnerability:** It's difficult to track when keys were rotated, if a rotation failed, or who initiated it.
 - **Mitigation:** The script should log extensively to a dedicated log file or a centralized logging system (e.g., Syslog, ELK stack). Ansible Tower / AWX provides a comprehensive audit trail out-of-the-box.

3.3. Optimizations & Scalability Improvements

- **Centralized Key Management:** For environments with hundreds or thousands of servers, managing individual keys in `authorized_keys` becomes untenable.
 - **Optimization:** Implement an **SSH Certificate Authority (CA)** using `ssh-keygen` or a dedicated system like **HashiCorp Vault (SSH Secrets Engine)** or **Teleport**.
 - **Architecture:**
 1. A central CA holds a master key pair.
 2. Servers are configured to trust any key signed by this CA (`TrustedUserCAKeys` directive in `sshd_config`).
 3. Users/services request short-lived certificates from the CA (e.g., valid for 8 hours) instead of permanent public keys.
 - **Advantages:**
 - **No `authorized_keys` management:** Servers no longer need a list of public keys.
 - **Automatic Expiration:** Keys become invalid automatically when their certificate expires. "Rotation" is continuous and implicit.
 - **Centralized Auditing & Control:** All access is brokered and logged by the CA. Access can be revoked instantly at the CA level.